



SQL in Production

Notebooks as an IDE

Please do not copy without permission. © School of Entrepreneurship and Technology.



What Are IDEs and Python Notebooks?

Integrated Development Environments (IDEs) are a collection of tools used for **software development**, while Python **notebooks** are a collection of tools for interactive **data analysis** and **visualization**.



IDEs

Text editor interface for coding, debugging, and testing.

They often support **multiple languages** and come with built-in tools for **version control** and collaboration.

Additional **plugins** support software development tasks, including **building**, **compiling**, and **deploying** applications.



Notebooks

Interactive interface consisting of **cells** used for data analysis, visualization, and documentation.

Libraries are imported for a range of functionalities, including **visualization**, **modeling**, and more.

Supports the development of **tutorials**, **research**, and **storytelling** through data.



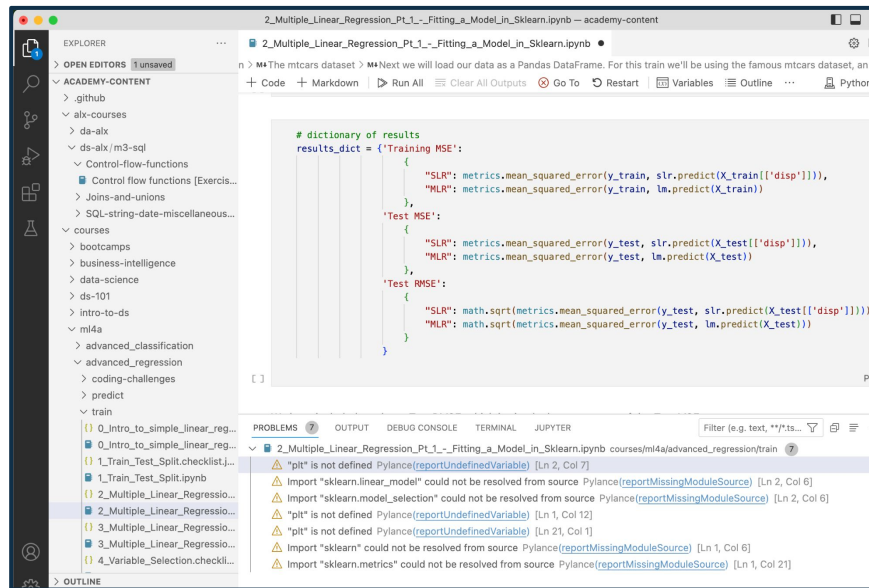
IDEs

IDEs are a set of ‘traditional’ tools that help users develop software.

An IDE, or Integrated Development Environment, is a **digital toolkit for computer programmers**. It provides an engaging and organized environment for **writing, testing, and managing code**.

They are a one-stop shop where programmers can **create their code, edit it, debug it**, and even **run their programs**, all in one convenient location.

They often also incorporate functionality such as **code editors, debugging tools**, and **project management capabilities** to make the software development process more efficient.

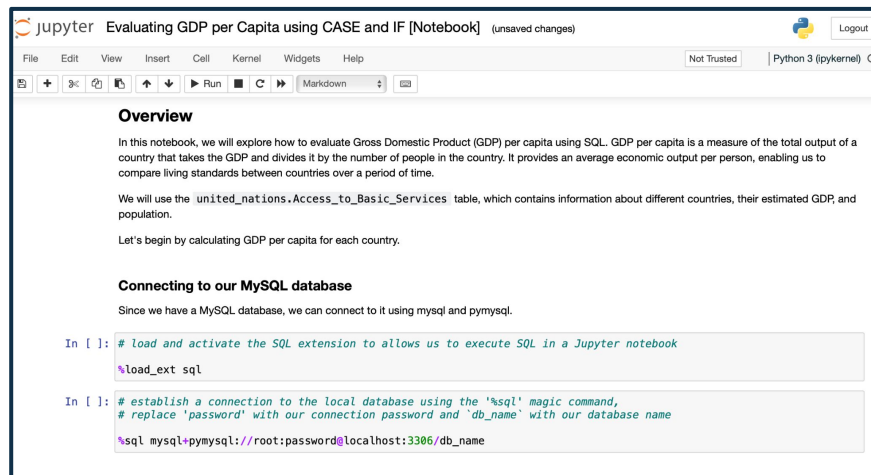




Notebooks

Notebooks are used for **processing and visualizing data**, where sections of code can be executed separately, with supporting text to describe what is happening.

In the context of coding, a notebook is similar to a **digital notepad** or **journal**. It is a software tool or platform that enables you to **create and organize documents that contain a combination of text, code, and visual elements**. These notebooks are often employed for **data analysis, scientific research, and interactive coding projects**. Think of a notebook as a **versatile digital notebook** where you can **write notes, perform calculations, run code, and visualize data**, all in one place.





Notebooks vs IDEs

Notebooks and IDEs

Both notebooks and IDEs are tools for **creating** and **running code**, with IDEs focusing on traditional software development tools while notebooks are geared towards data analysis, research, and interactive documentation.

The **choice** between them is determined by the **individual task** and **user requirements**.

Synergy

In a software development project, **IDEs** and **notebooks** can **complement** one another. **Notebooks** can be used for **data exploration** and **prototyping** to quickly test ideas and visualize results, as well as for documenting thought processes, experiments, and findings.

IDEs can be used for **rigorous testing and debugging** of software components, and their version control features can be used to **manage code revisions and collaboration**.



Notebook Environments

Anaconda, Jupyter Notebook, and Google Colab are **tools** that streamline **data analysis** in interactive notebook environments.

Jupyter Notebook is an open-source **web application** for creating documents with code, visualizations, and text.

Jupyter
Notebook

Google Colab is a **cloud-based platform** provided by Google that allows users to write, run, and share code in an interactive and collaborative environment.

Google
Colab

VS Code (Visual Studio Code) is a **versatile desktop application** that can be used for code development and data exploration with notebooks

VS Code



Notebook
Environments



Jupyter Notebook

Jupyter Notebook runs as a **web application** where we can **mix code, visuals, and text** in one document. It allows us to generate and collect **real-time results**, making learning and experimentation very intuitive.

01. Code

We can **write** and **execute code** in **small, manageable** chunks called cells, making it easier to experiment and iterate on ideas.

02. Text

These cells can be interspersed with **rich-text** explanations, allowing us to provide **context, documentation, and insights** alongside our code.

03. Visualizations

By embedding charts, graphs, and interactive plots alongside our code, we can **illustrate our findings and convey complex information** in an accessible manner.

The screenshot displays a Jupyter Notebook titled "Control flow functions [Exercise] (autosaved)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and markdown editing. The notebook content is organized into three main sections:

- Learning objectives:** A text block stating "In this train we will learn to:" followed by two bullet points:
 - Use the `IIF` control function to check for a certain condition and return a value for each scenario.
 - Use the `CASE` statement to categorise data in an existing column.
- Code cells:** Two code cells are shown. The first cell (In [1]) contains SQL code to load and activate the SQL extension. The second cell (In [2]) contains SQL code to load the Northwind database and specify the SQLite file path.


```

      In [1]: 1 # load and activate the SQL extension to allows us to execute SQL in a Jupyter notebook
      2 %load_ext sql

      In [2]: 1 # load the Northwind database stored in your local machine.
      2 # Make sure the file is saved in the same folder as this notebook.
      3 %sql sqlite:///Northwind.db
      
```
- Database Schema:** A diagram showing the relationships between tables in the Northwind database. The tables include Customers, CustomersDemo, Shippers, Employees, Orders, and OrderDetails. Each table is represented by a box containing its columns and data types.
 - Customers:** CustomerID (VARCHAR(5)), CompanyName (VARCHAR(40)), ContactName (VARCHAR(30)), ContactTitle (VARCHAR(30)), Address (VARCHAR(60)), City (VARCHAR(15)), Region (VARCHAR(10)), PostalCode (VARCHAR(10)), Country (VARCHAR(15)), Phone (VARCHAR(24)), Fax (VARCHAR(24)).
 - CustomersDemo:** CustomerID (VARCHAR(5)), CustomerTypeID (VARCHAR(10)).
 - Shippers:** ShipperID (INT), CompanyName (VARCHAR(40)), ContactName (VARCHAR(30)), ContactTitle (VARCHAR(30)), Address (VARCHAR(60)), City (VARCHAR(15)), Region (VARCHAR(10)), PostalCode (VARCHAR(10)), Country (VARCHAR(15)), Phone (VARCHAR(24)), Fax (VARCHAR(24)), HomePage (MEDIUMTEXT).
 - Employees:** EmployeeID (INT), CompanyName (VARCHAR(40)), ContactName (VARCHAR(30)), ContactTitle (VARCHAR(30)), Address (VARCHAR(60)), City (VARCHAR(15)), Region (VARCHAR(10)), PostalCode (VARCHAR(10)), Country (VARCHAR(15)), Phone (VARCHAR(24)), Fax (VARCHAR(24)), HomePage (MEDIUMTEXT).
 - Orders:** OrderID (INT), CustomerID (VARCHAR(5)), EmployeeID (INT), OrderDate (DATETIME), RequiredDate (DATETIME), ShippedDate (DATETIME), ShipVia (INT), Freight (DECIMAL(10,4)), ShipName (VARCHAR(40)), ShipAddress (VARCHAR(60)), ShipCity (VARCHAR(15), ShipRegion (VARCHAR(10)), ShipPostalCode (VARCHAR(10), ShipCountry (VARCHAR(15)).
 - OrderDetails:** OrderID (INT), ProductID (INT), Quantity (INT), UnitPrice (DECIMAL(10,4)), Discount (DECIMAL(10,4)), ShipVia (INT), ShipDate (DATETIME), ShipRegion (VARCHAR(10), ShipPostalCode (VARCHAR(10), ShipCountry (VARCHAR(15)).



Google Colab

Google Colab is a **cloud-based** platform where you can write and run code. It offers free access to powerful machines (**virtual servers**) and integrates easily with **Google Drive** as a data source.

01. Powerful Computing and Storage

High-performance servers are available for **data processing** and **modeling**, with seamless Google Drive integration.

02. Collaborative Environment

Enables **real-time collaboration** with team members through shareable links, comments, and more collective editing.

03. Enhanced Coding Experience

The workspace has some **IDE-like features**, including autocomplete, syntax highlighting, a variable inspector, and the ability to format visualizations.

The screenshot displays the Google Colab interface for a notebook titled "CTE vs Subquery [Notebook].ipynb". The interface is divided into several sections:

- Table of Contents:** Located on the left, it lists sections like "Common Table Expressions (CTEs) vs. Subqueries", "Learning Objectives", "01.", "database", "Exercise", "Task 1: Collect the columns", "Task 2: Get the base query", "Task 3: Implement the solution using subqueries", "Task 4: Implement the solution using Common Table Expressions (CTEs)", "Solutions", "Task 1: Collect the columns", "Task 2: Get the base query", "Task 3: Implement the solution using subqueries", "Task 4: Implement the solution using Common Table Expressions (CTEs)", "Summary", and "Section".
- Main Workspace:** The central area contains text and code. It includes a section titled "will include an additional calculated column:" followed by a paragraph about data collection. Below this is a code cell with a SQL query:

```
SELECT column1, column2, column3, column4, column5, column6 FROM database.table;
```

 The column names in the query are highlighted in blue.
- Right Sidebar:** Contains a "Connect" button and a "Variable Inspector" section.

Overlaid on the screenshot are three numbered boxes:

- 01.** A green box highlighting the "Learning Objectives" section in the Table of Contents.
- 02.** A blue box highlighting the "Task 2: Get the base query" section in the Table of Contents.
- 03.** A cyan box highlighting the SQL query in the code cell.



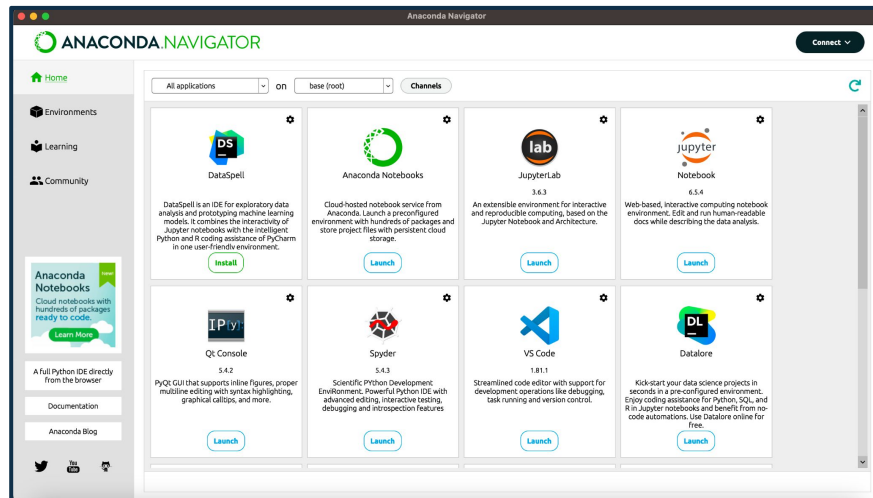
Anaconda

Anaconda is an **all-in-one** data science toolkit that bundles together many tools for data analysis and science.

Anaconda Navigator is a Graphical User Interface (GUI) included with the Anaconda distribution. It is designed to help users **manage** and **interact** with their Anaconda **environments**, **packages**, and **projects**.

It comes **pre-packed** with over 1,500 tools and **platforms** such as Jupyter, Pandas, and Matplotlib, which helps with the use and management of these tools in one central place.

Anaconda can be installed on different operating systems (e.g., Mac or Windows) and environments (a computer or cloud provider), making it **platform independent**.





Notebook Cell Types

Markdown Cells: Used for documentation. We can write text, add headers, and format the content to explain code.

Code Cells: Used for input commands and scripts. When executed, the notebook processes the code and displays the output.

Raw Cells: Unprocessed content. It's a space to input data or notes without the notebook interpreting or executing it.

jupyter Cell_Types Last Checkpoint: 2 minutes ago (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

Run

Overview

Continuing with the numerical analysis of our Access_to_Basic_Services table created in MySQL Workbench, we want to filter and analyse our already created summary statistic report. We can apply the same queries in MySQL Workbench and in this notebook if we connect to our MySQL server.

We want to use the summary statistic report to do the following:

1. Filter for the year 2020.
2. Focus on countries where the percentage of managed drinking water services is below 60%.
3. Filter for the regions and sub-regions that have fewer than four countries.

```
In [ ]: %%sql
SELECT column1,
       column2,
       MIN(column3) AS min_column3,
       MAX(column3) AS max_column3,
       AVG(column3) AS avg_column3,
       COUNT(DISTINCT(column1)) AS column1_categories,
       SUM(column4) AS column4_total
FROM database.table
GROUP BY column1, column3
ORDER BY column4_total;
```

Note for later:

- Check data sources for GDP figures.
- Update the database password for security reasons.
- Consider adding more economic indicators for a comprehensive analysis.
- Remember to replace 'db_name' with the actual name of our database before executing.



Other Programming Languages Used in Notebooks

Notebook environments like Jupyter Notebook and JupyterLab **support a variety of programming languages, not just SQL**. Some commonly supported programming languages in Jupyter Notebook include

Python: One of the most popular languages for data science and general-purpose programming used for data analysis, machine learning, and scientific computing.

R: Commonly used for statistical analysis, data visualization, and machine learning.

JavaScript: Useful for web development and interactive data visualization tasks.

These are just some examples of the many languages we can use in Jupyter Notebook. This versatility makes Jupyter Notebook a powerful tool for various programming and data analysis tasks, and it enables **polyglot programming**, where we can combine **multiple languages in a single notebook**.

Notebooks help us **combine the capabilities** of SQL, Python, and other languages to perform data exploration and analysis.

